Master Degree Computer Engineering
Computer Architecture

# String Comparator

Megan Maremmani
Eleonora Sgorbini
Sebastiano Pala

0110
1001
1010 ALGORITHM

- **The idea**
- **How it works**

 GOALS

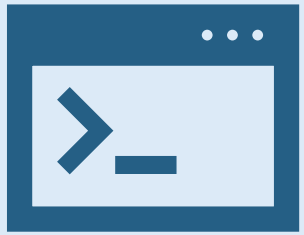
CPU

 GPU

# ALGORITHM: The idea

The purpose of the algorithm is to **count how many times a target word appears in a text file.**

**Let's see how it works!**



EXAMPLE:

*This is my dog. His name is **Max**. **Max** is a good dog. **Max** is very happy. He likes to run. He runs in the park. He runs in the garden. He runs every day. **Max** likes to play with his ball. The ball is red. It is a big red ball. **Max** runs to the red ball. He plays with the ball in the morning. He plays with the ball in the afternoon.*

EXPECTED OUTPUT:

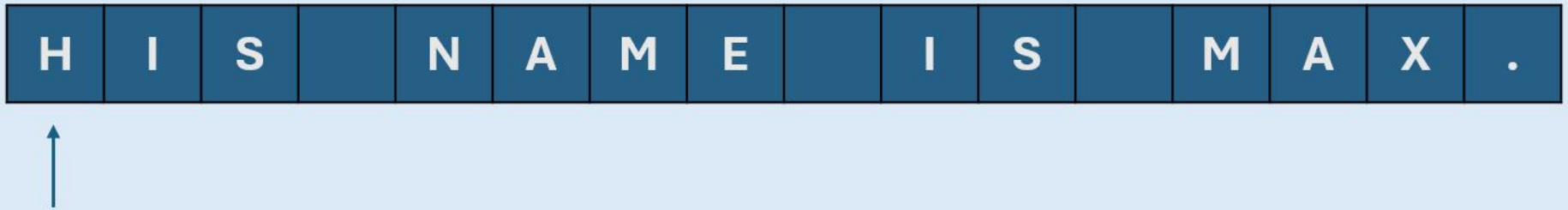
*"Max" occurrences: 5*

# ALGORITHM: How it works

## ITERATION

TARGET STRING:


TEXT:




ALGORITHM

GOALS

- **The main objective**


CPU

GPU

# GOALS

The primary objective is to evaluate the maximum throughput while determining the number of instances of the specified string

## USER POV

Throughput should be at least 5 GBytes/s for large files (over 4 GBytes)

## DEVELOPER POV

To be continued...


ALGORITHM

GOALS


CPU

GPU

- **Intel i7 CPU and tools**
- **Naive version**
- **KMP**
- **KMP with load balancing**

# Intel i7 240H CPU

## HARDWARE DETAILS

**Performance-cores x threads per core**
6 x 2 = 12

**Efficient-cores x threads per core**
4 x 1 = 4

**Logical cores**
12 + 4 = 16

| Cache level | P-Core | E-core |
|---|---|---|
| L1 - Data | 48 KB (per core) | 32 KB (per core) |
| L1 - Inst | 32 KB (per core) | 64 KB (per core) |
| L2 | 1.25 MB (per core) | 2MB (shared) |
| L3 | 24MB | (shared) |

## CPU

# TOOLS:

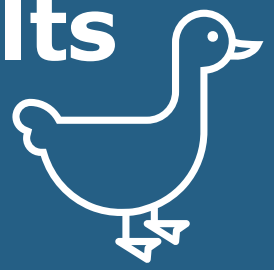
**Intel VTune Profiler**

**Visual Studio code**

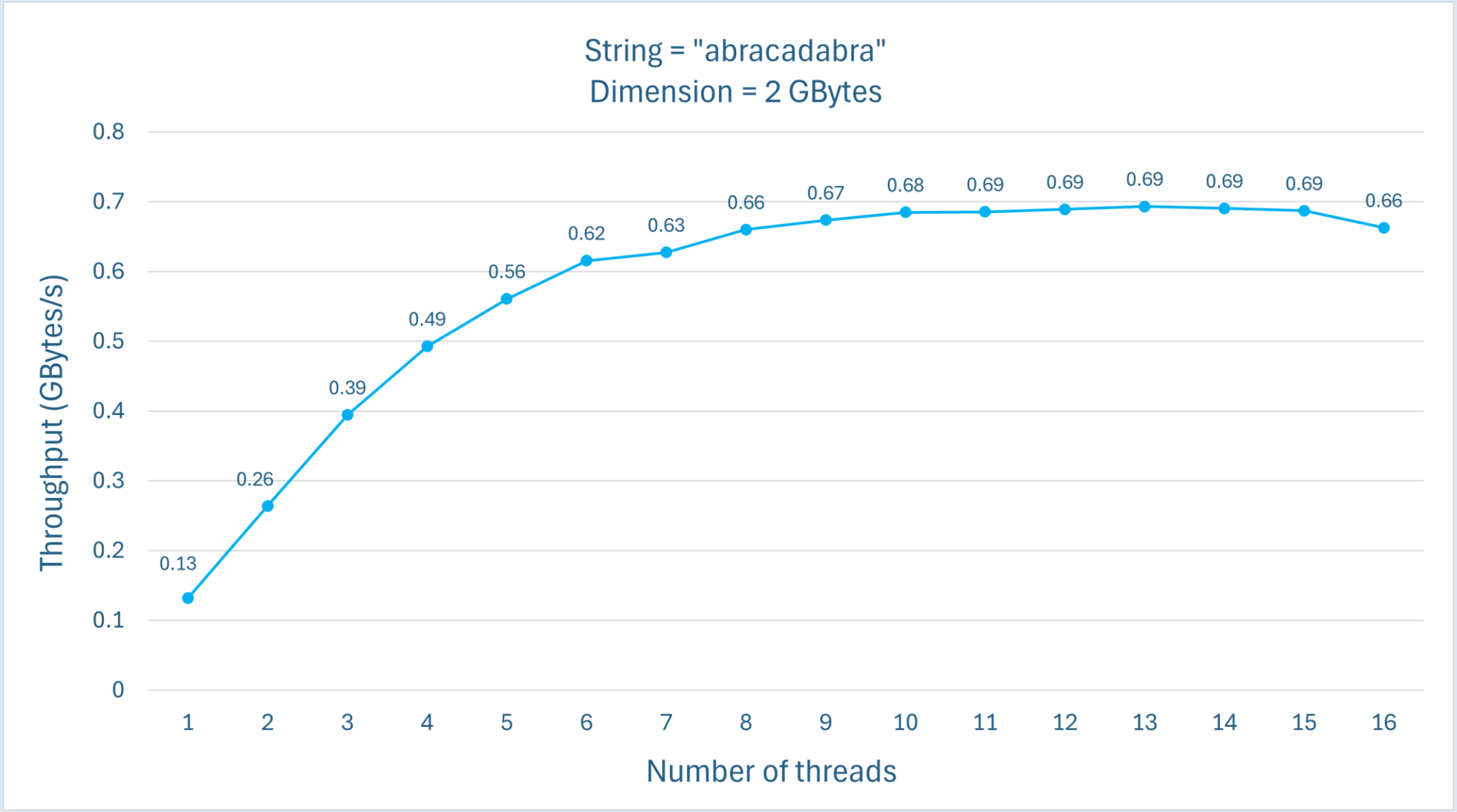
**Chrono libraries for timing**

```
chrono::steady_clock::time_point start = chrono::steady_clock::now();
// algorithm execution
chrono::steady_clock::time_point end = chrono::steady_clock::now();

// compute overall duration
chrono::milliseconds duration = chrono::duration_cast<chrono::milliseconds>(end - start);
```

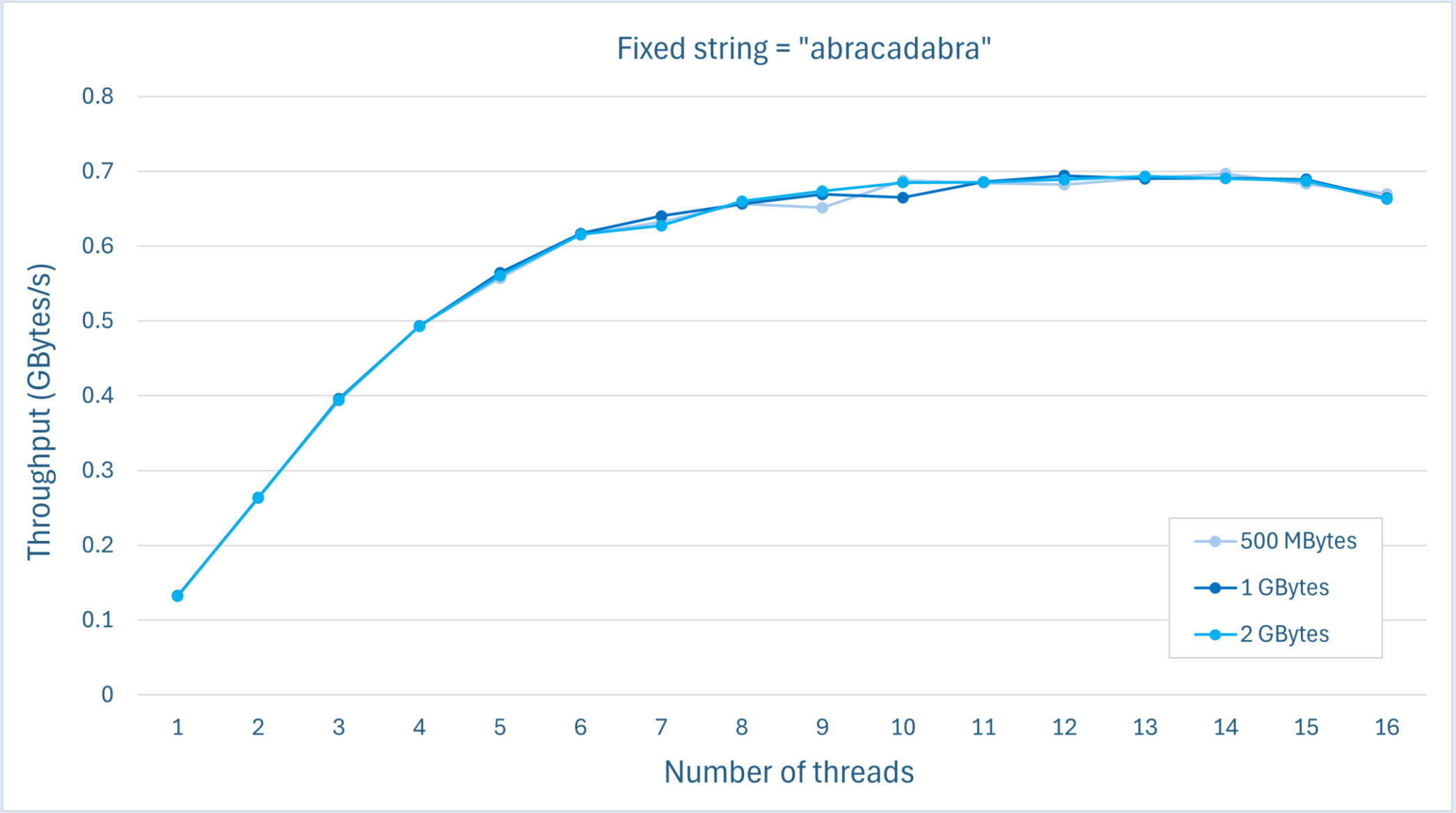
# Let’s see some results about the first implementation



# Throughput analysis: fixed string

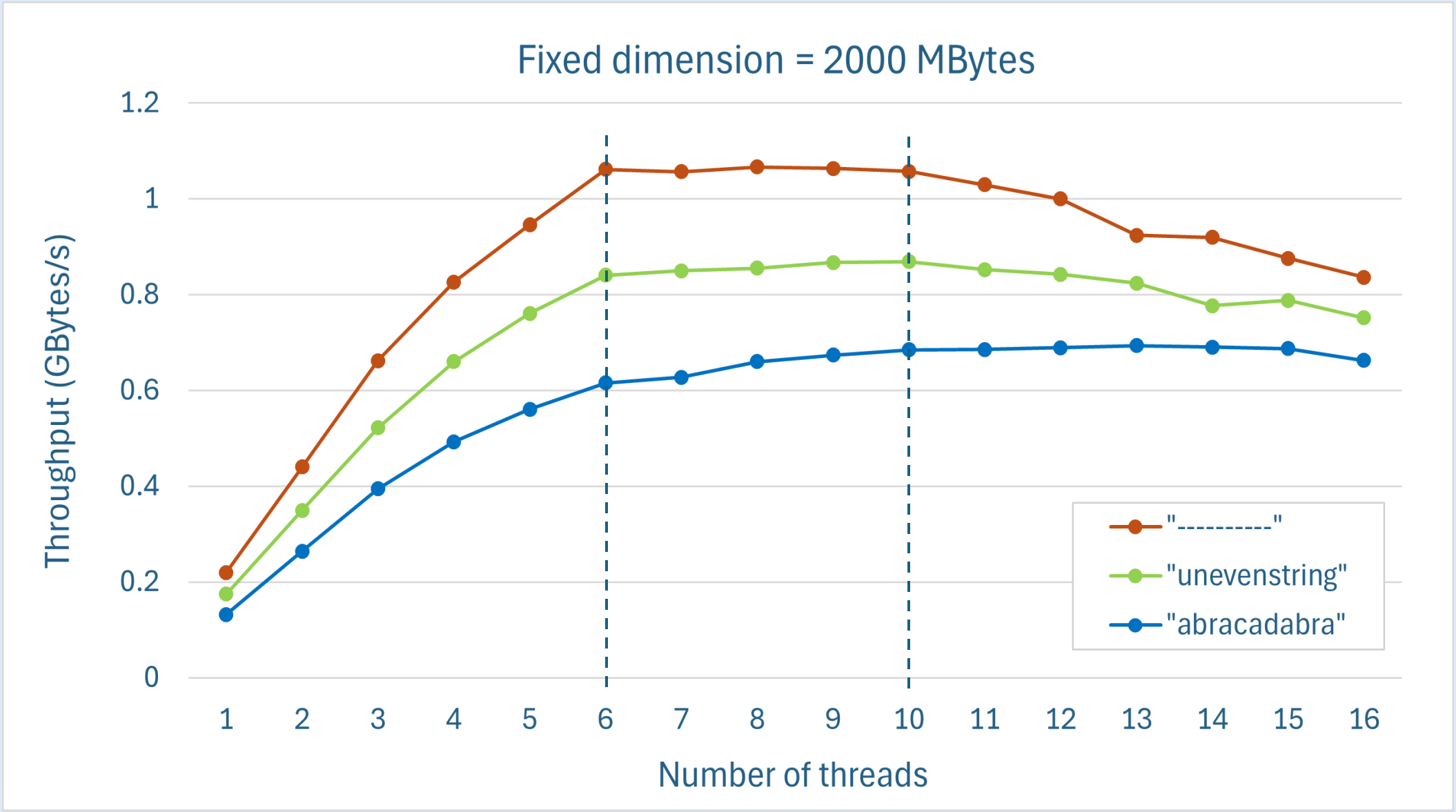


# Throughput analysis: fixed string

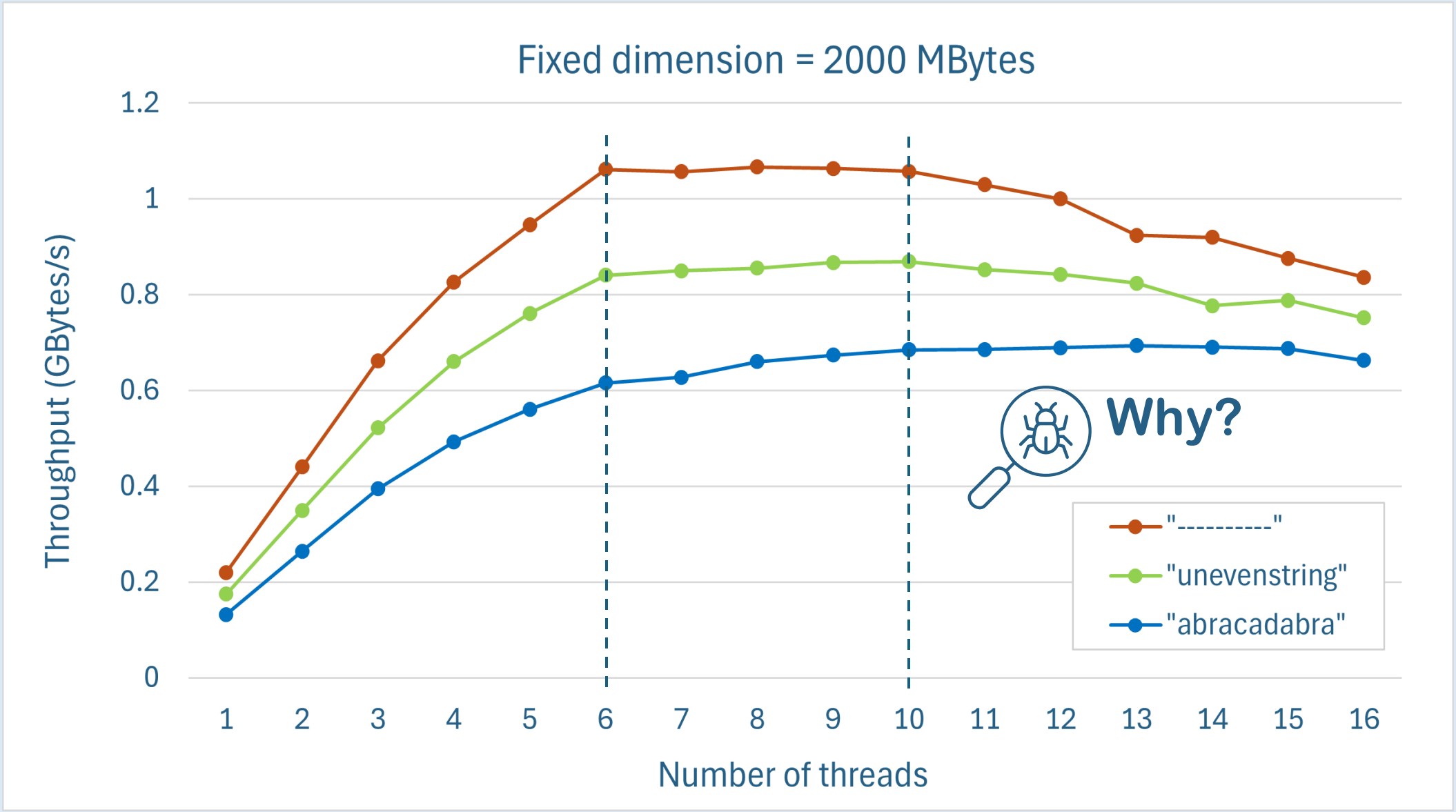


# But… what about different words?

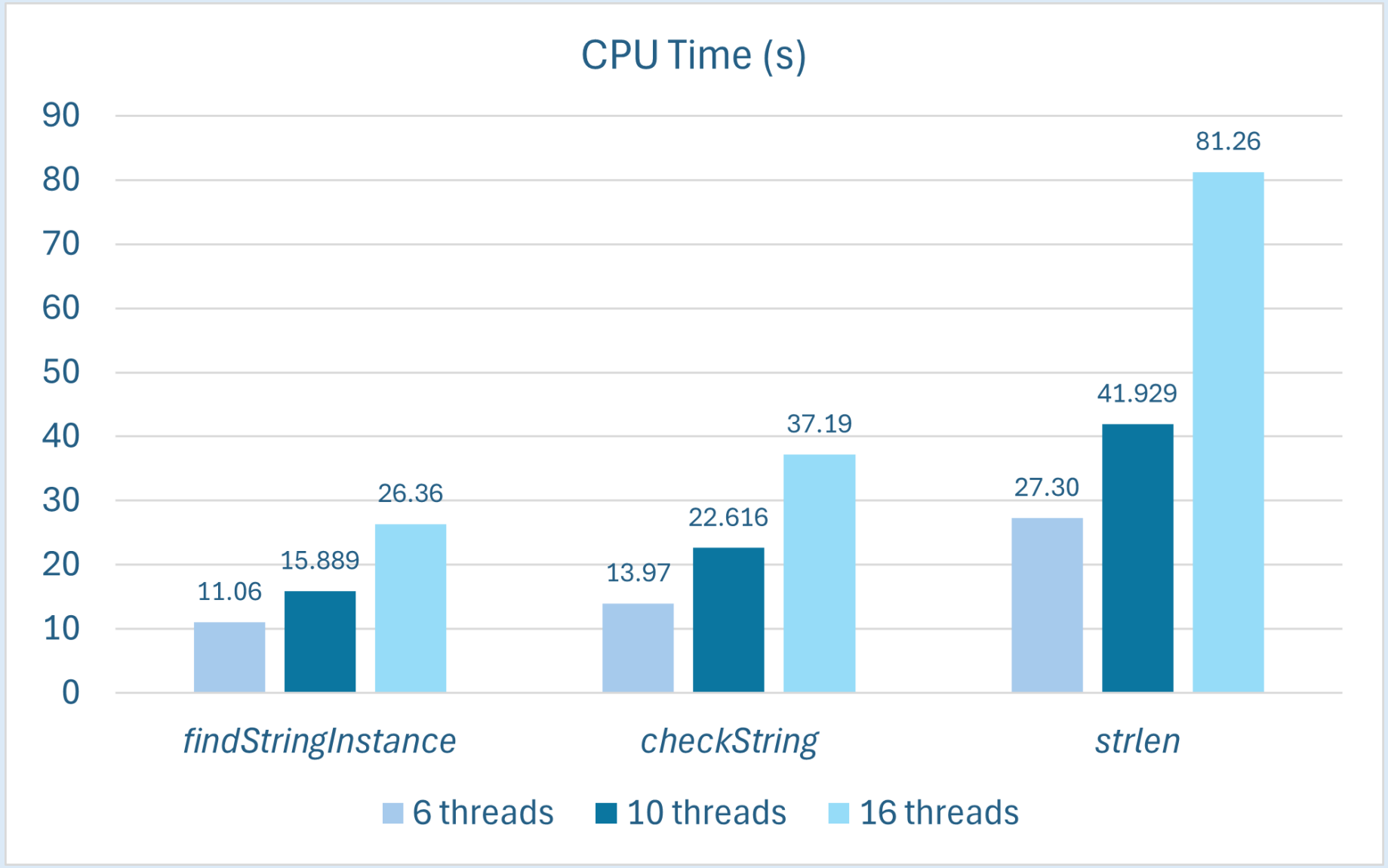
# Throughput analysis: fixed dim



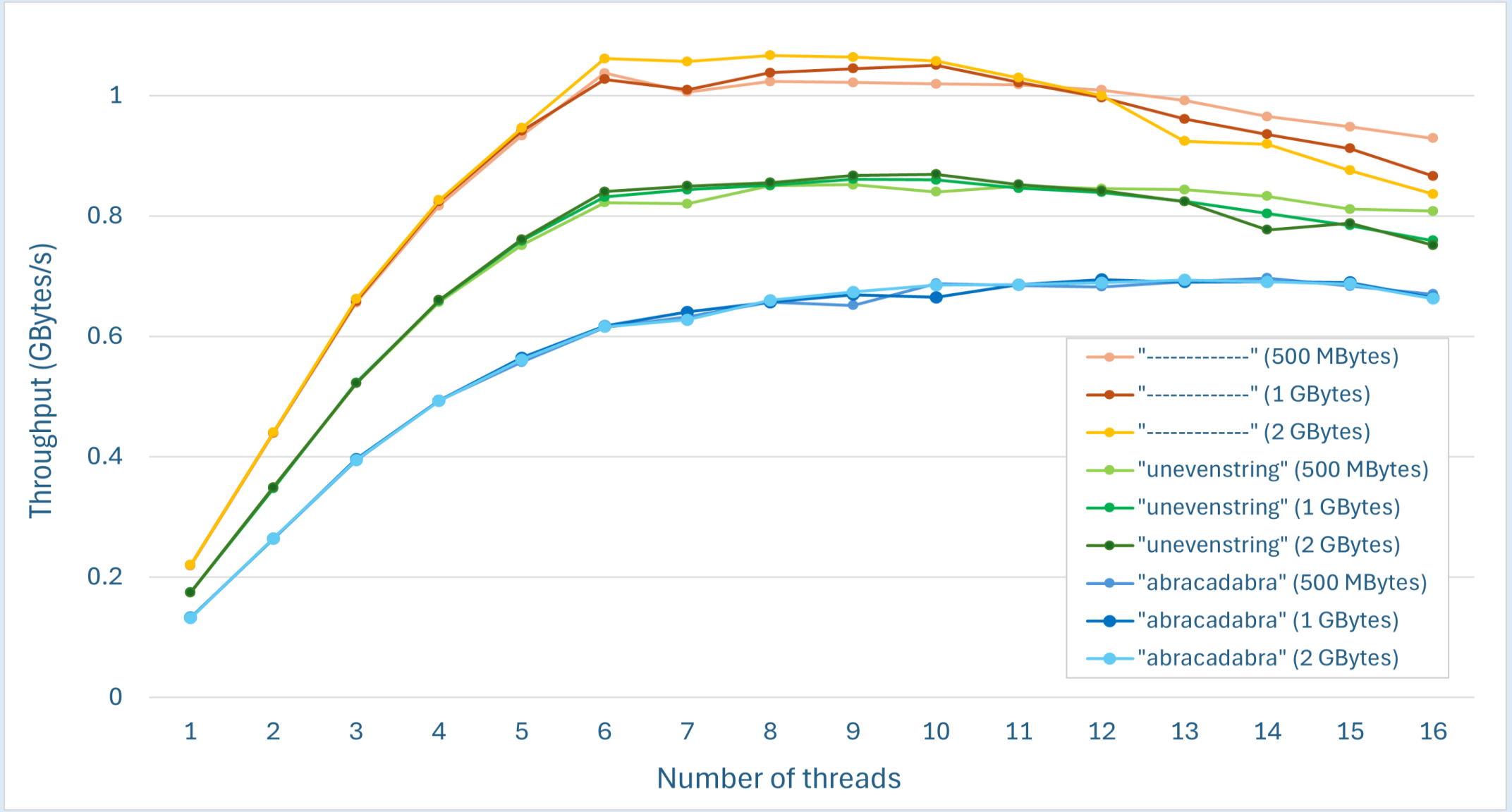
# Throughput analysis: fixed dim



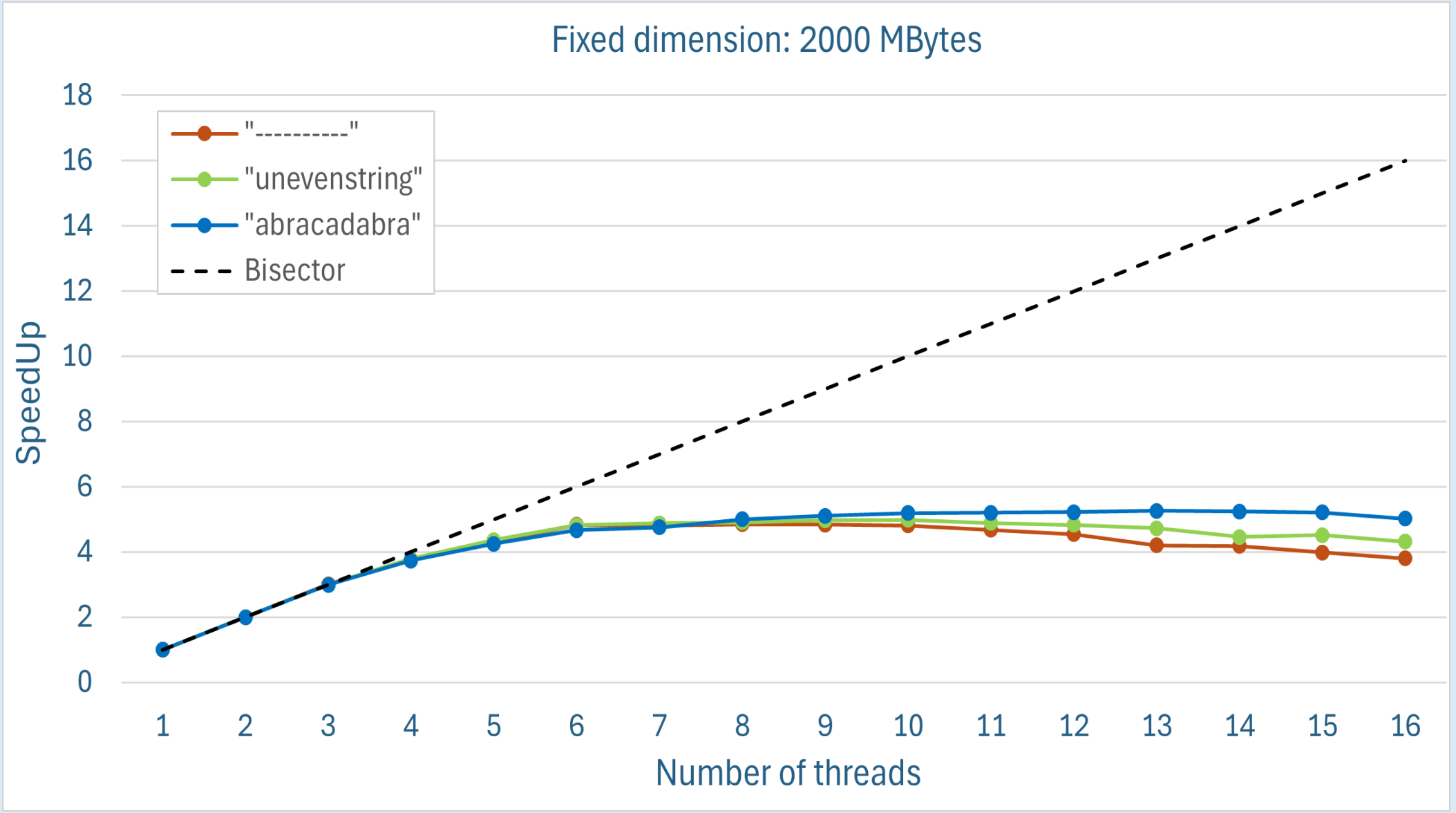
# Focus


CPU Time (s)
90
80
70
60
50
40
30
20
10
0
11.06
15.889
26.36
13.97
22.616
37.19
27.30
41.929
81.26
findStringInstance
checkString
strlen
6 threads
10 threads
16 threads

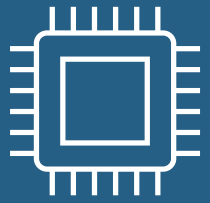
# Throughput analysis: comparison



# SpeedUp analysis: fixed dimension



# Is it possible to do better?



0110
1001
1010

# Hot Spots


| Function | CPU Time | % of CPU Time |
|---|---|---|
| *strlen* | 71.066 s | 39.1 % |
| *checkString* | 57.149 s | 31.5 % |
| *findStringIstance* | 40.663 s | 22.4 % |
| *Pthread_mutex_unlock* | 2.703 s | 1.5 % |

***The hot spot is strlen()!***

# The bottle neck: strlen()

*Function to check if the chars match the target word. Returns the number of chars that match, if it is equal to the length of the target string then we have found an occurrence*

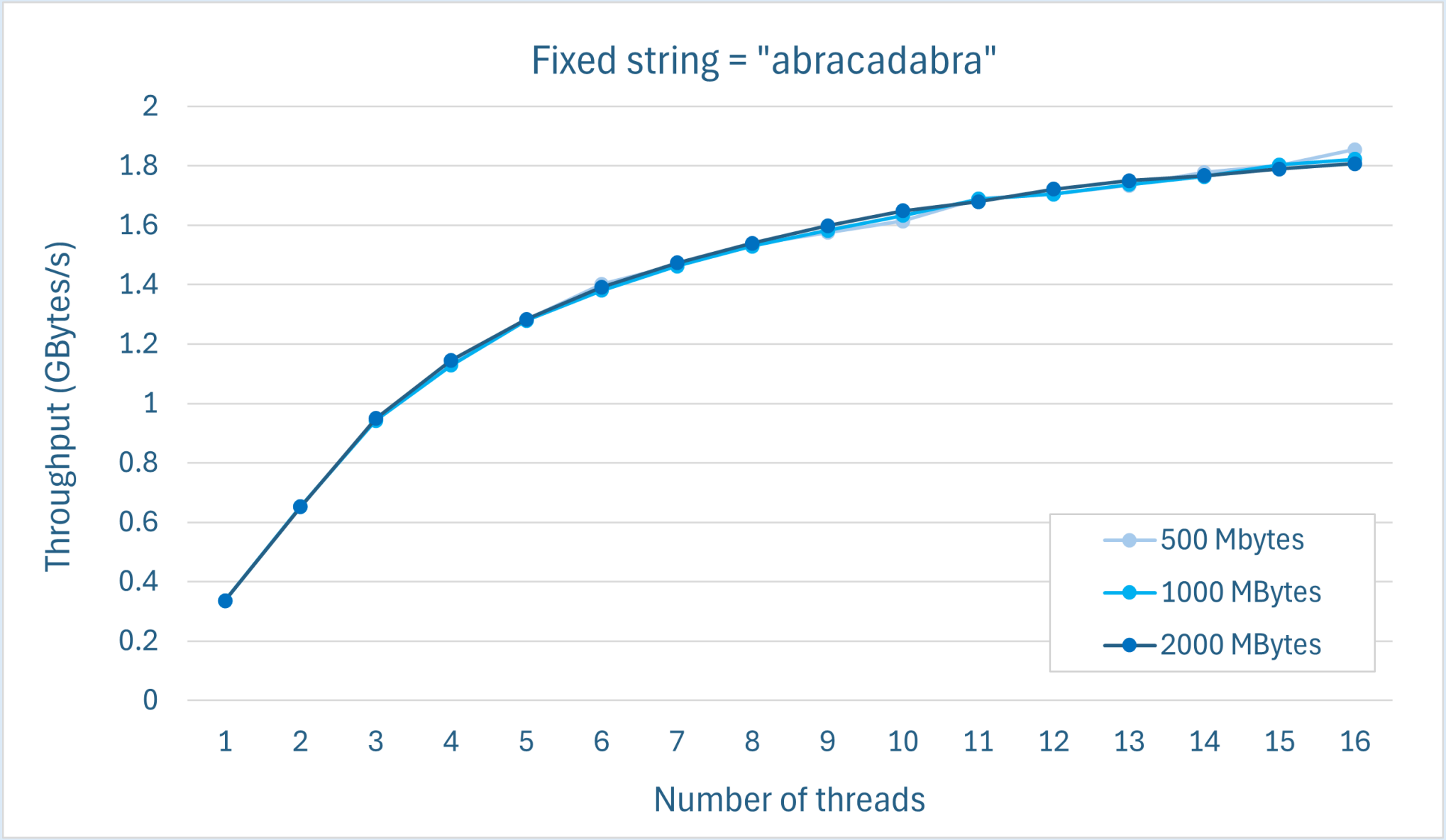
```
unsigned long checkString (char* candidate_match, char* target){

    unsigned long j;

    for (j = 0; j < strlen(target); j++){
        if (candidate_match[j] != target[j] ||
                (candidate_match + strlen(target) >= end_of_file)){
            break;
        }
    }
    return j;
}
```

# The bottle neck: strlen()

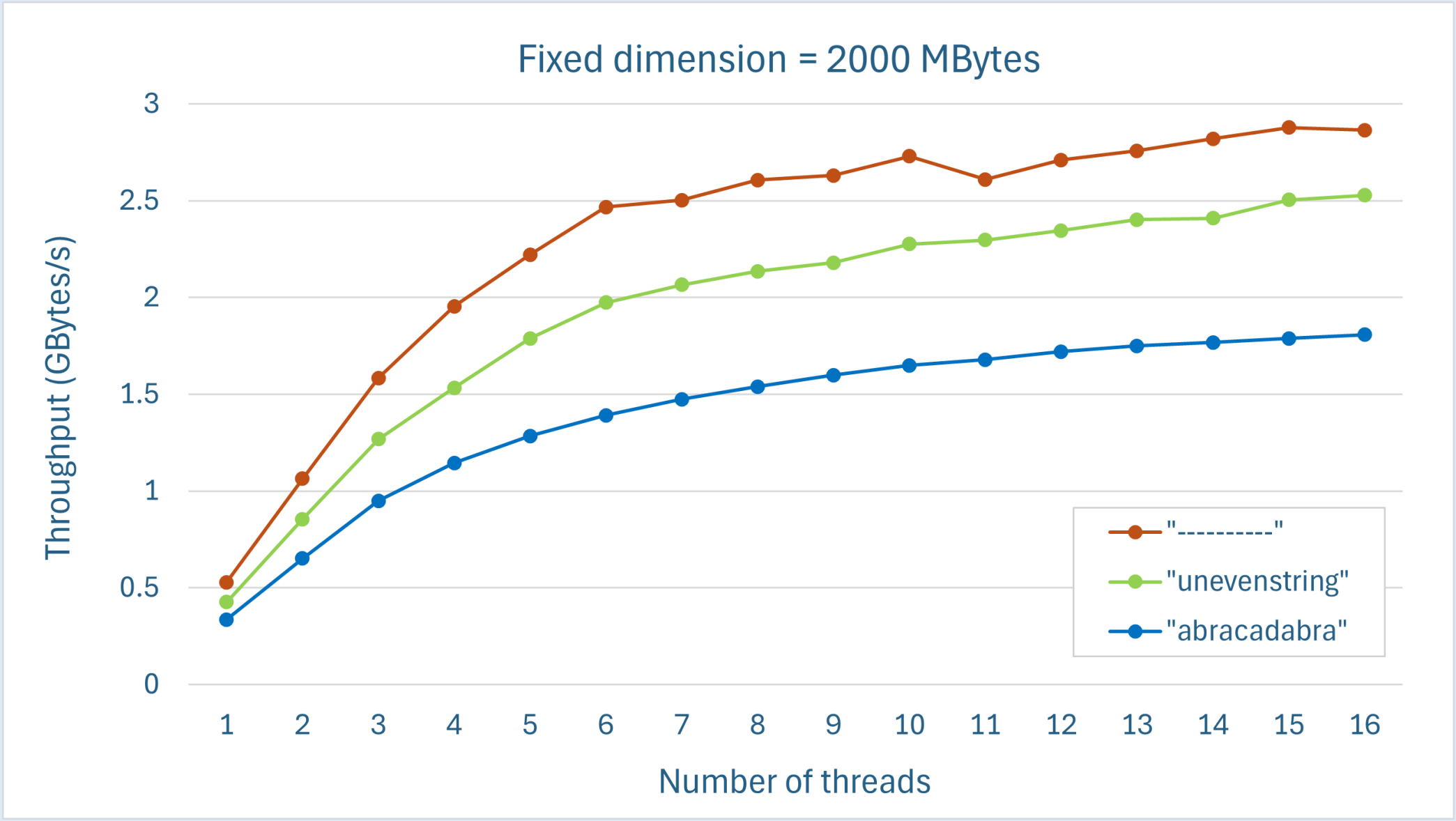
*The length of the target string is now computed only once in the main function and passed as an argument to the 'checkString' function*

```
unsigned long checkString (char* candidate_match, char* target, int
target_string_len){

    unsigned long j;

    for(j = 0; j < target_string_len; j++){
        if(candidate_match[j] != target[j]
                || (candidate_match + target_string_len >= end_of_file)){
            break;
        }
    }
    return j;
}
```

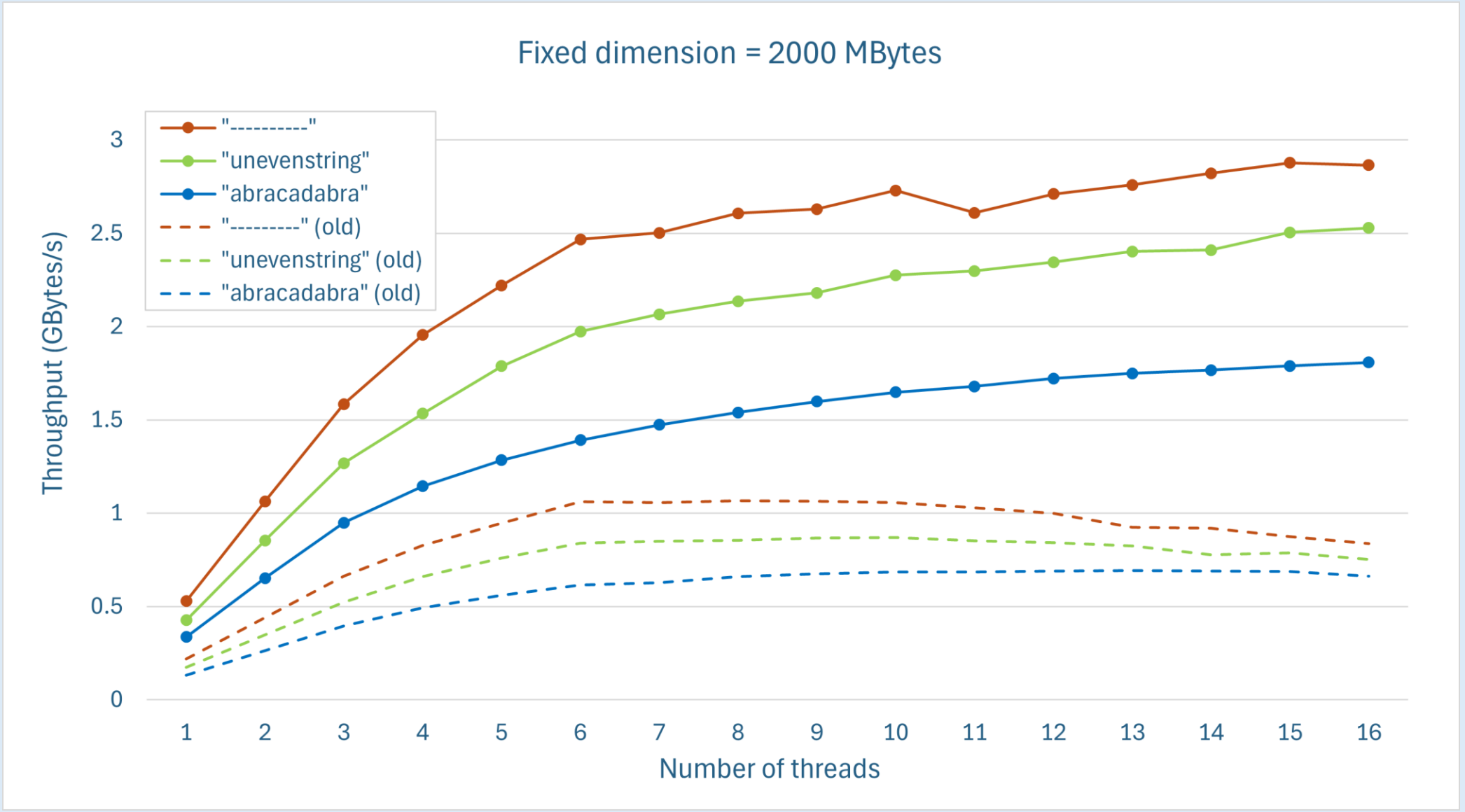
# Throughput analysis: fixed string



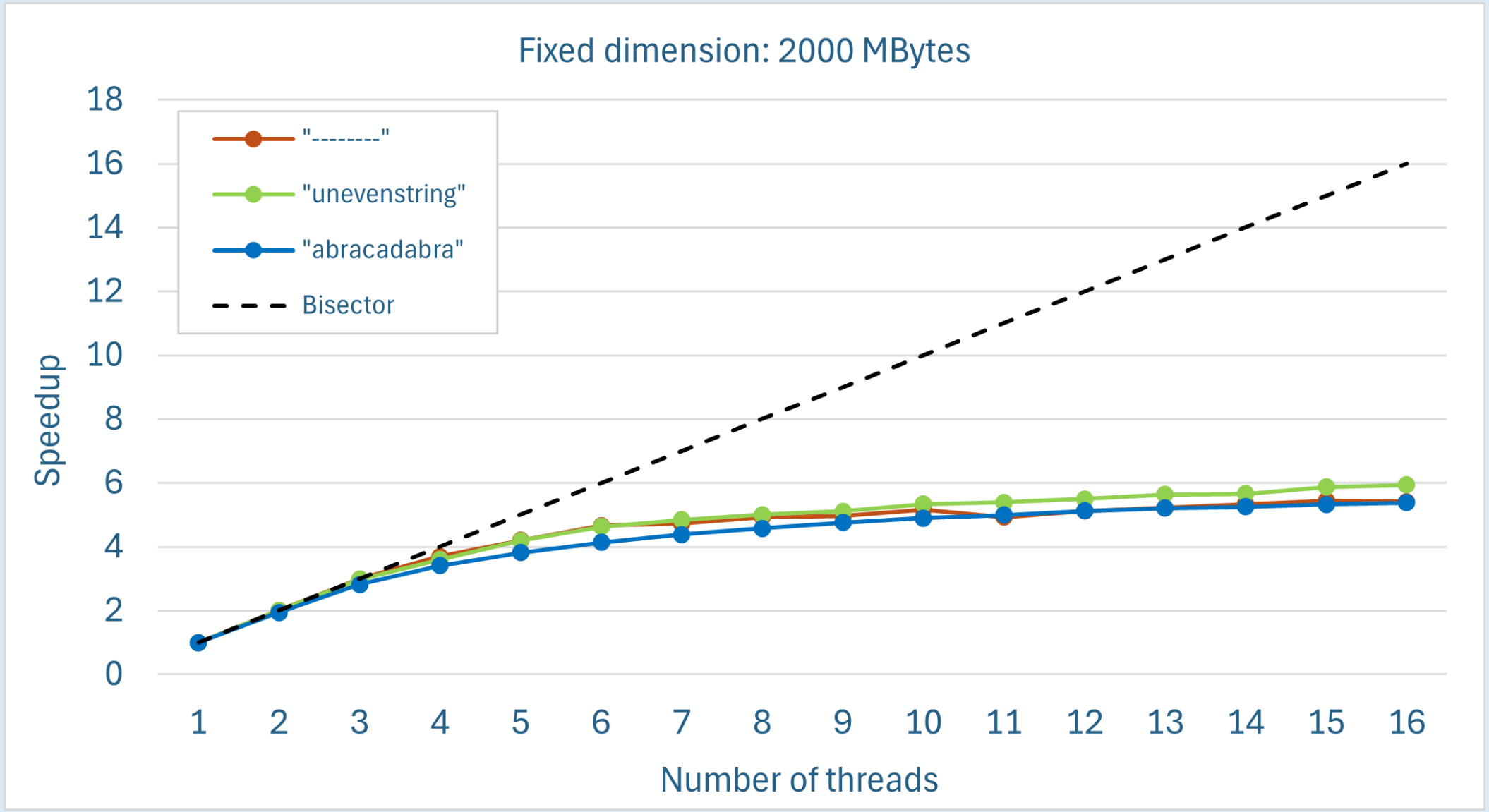
# Throughput analysis: fixed dim



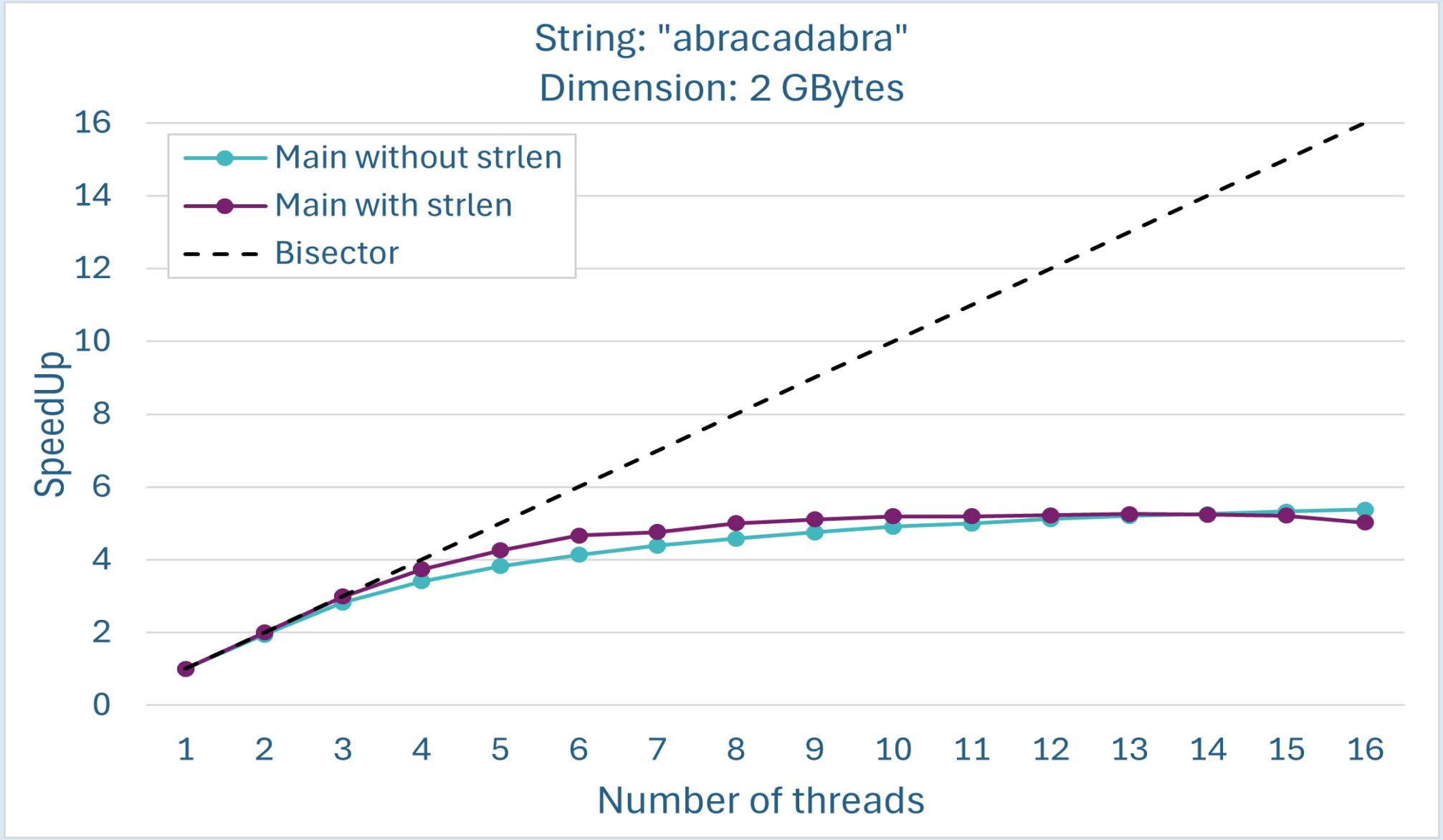
# Throughput analysis: comparison



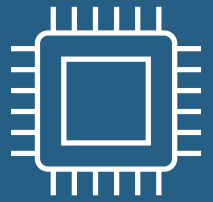
# SpeedUp analysis: fixed dimension



# SpeedUp analysis: comparison



# Can we do better?

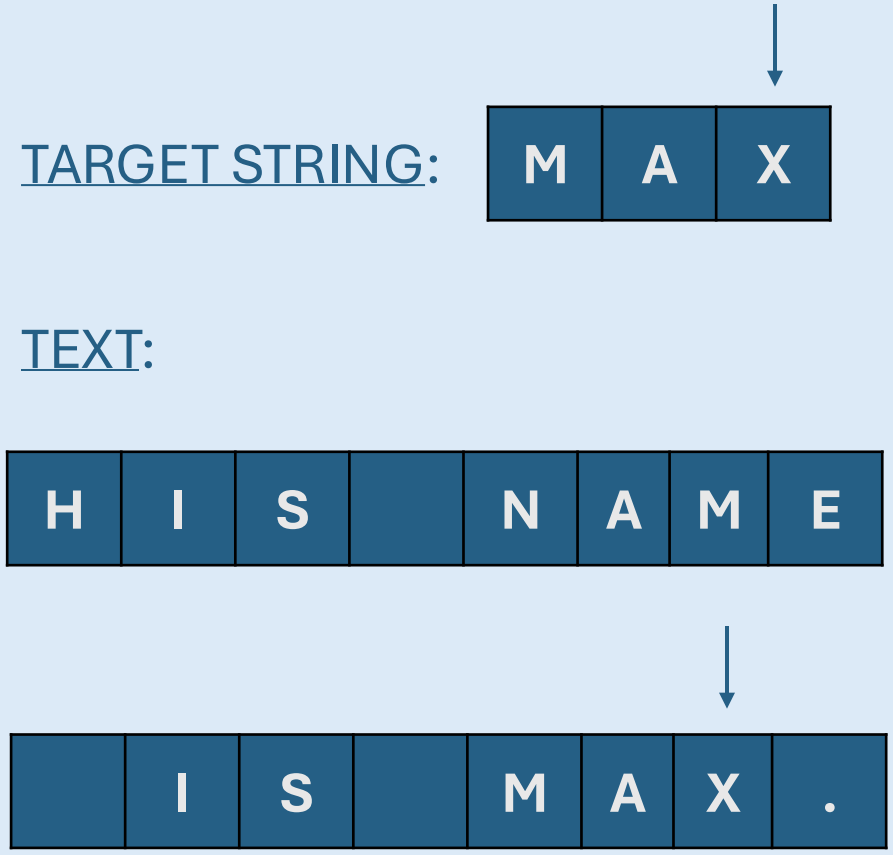


Where?

0110
1001
1010

# Can we do better? Where?

## ITERATION




**NOTE**

The second letter of our target word is different from the first. Therefore, if we find a match, we are certain that a new match won't start from the second letter.

**We are NOT using this property!**


# Let’s Optimize this code!

# KMP Algorithm

## FIRST STEP

We have to perform a pre-process of the target string in order to obtain what is called a **prefix-suffix table** (or the **longest prefix suffix**)

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | | |

```
void build_table (int len){
    char * head, *tail;
    head = tail = target_string;

    longest_prefix_suffix_array[0] = 0;
    tail++;
    int pos = 0;

    for(int i = 1; i < len; i++) {

        if (*tail == *head){
            pos++; head++;
            longest_prefix_suffix_array[i] = pos;
        } else {
            pos = 0; head = target_string;
            longest_prefix_suffix_array[i] = 0;
        }
        tail++;
    }}
```

# KMP Algorithm

## SECOND STEP

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

*j = 0*

*i = 0*

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
    // code
} else {
        if (j != 0)
            j = lps_array[j - 1];
        else
            i++;
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 0***

***i = 0***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
            local_occurrences++;
            j = lps_array [j - 1];
        }
} else {
   // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 0***

***i = 1***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
            local_occurrences++;
            j = lps_array [j - 1];
        }
} else {
   // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 1***

***i = 2***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 2***

***i = 3***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 3***

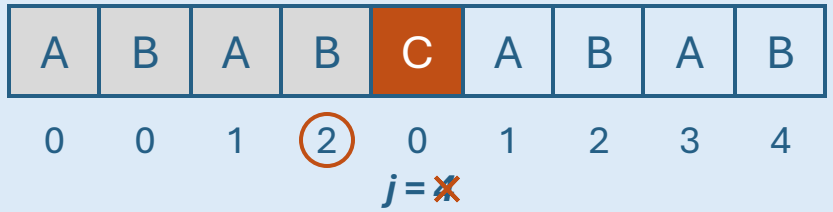
***i = 4***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
    // code
} else {
        if (j != 0)
            j = lps_array [j - 1];
        else
            i++;
}
```




*i = 5*

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
            local_occurrences++;
            j = lps_array [j - 1];
        }
} else {
   // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

*j = 2*

*i = 5*

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 3***

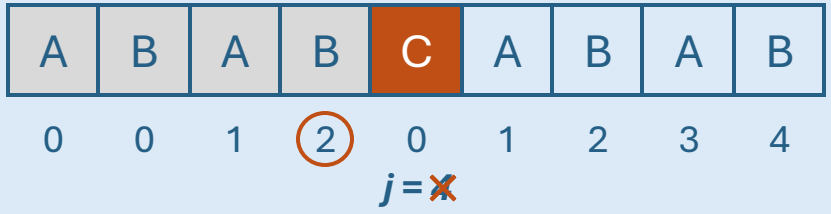
***i = 6***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
    // code
} else {
        if (j != 0)
            j = lps_array [j - 1];
        else
            i++;
}
```




*i = 7*

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 2***

***i = 7***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 3***

***i = 8***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 4***

***i = 9***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 5***

***i = 10***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 6***

***i = 11***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 7***

***i = 12***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
           j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

*j = ~~8~~*

*i = 13*

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
            local_occurrences++;
            j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

*j = 4*

*i = 14*

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
            local_occurrences++;
            j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 5***

***i = 15***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
            local_occurrences++;
            j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 6***

***i = 16***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
            local_occurrences++;
            j = lps_array [j - 1];
        }
} else {
  // code
}
```

| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

***j = 7***

***i = 17***

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# KMP Algorithm

## SECOND STEP

```
if (target_string[j] == file_buffer[i]) {
        j++;  i++;

        if (j == target_string_length){
           local_occurrences++;
            j = lps_array [j - 1];
        }
} else {
  // code
}
```

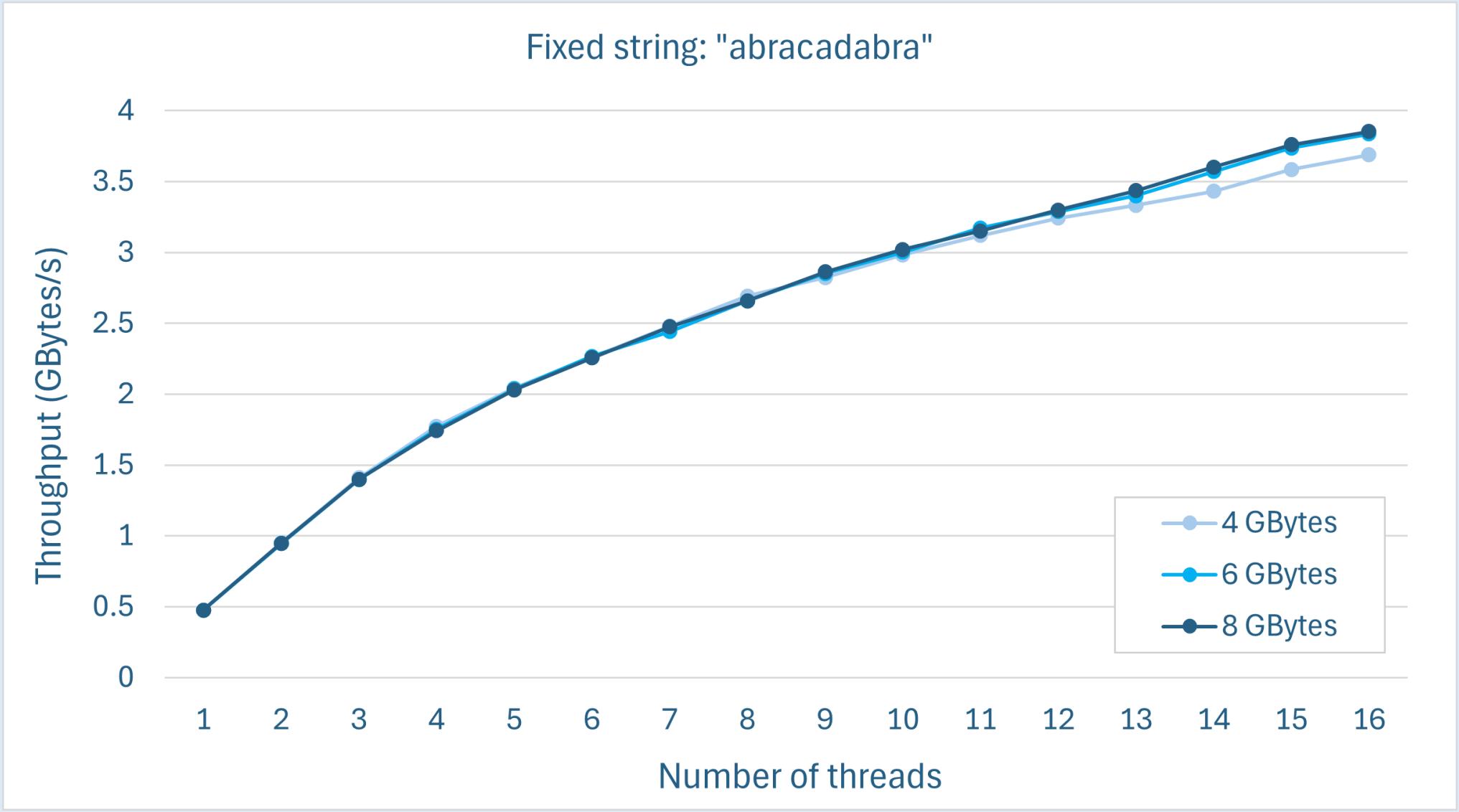
| A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

*j = ~~8~~*

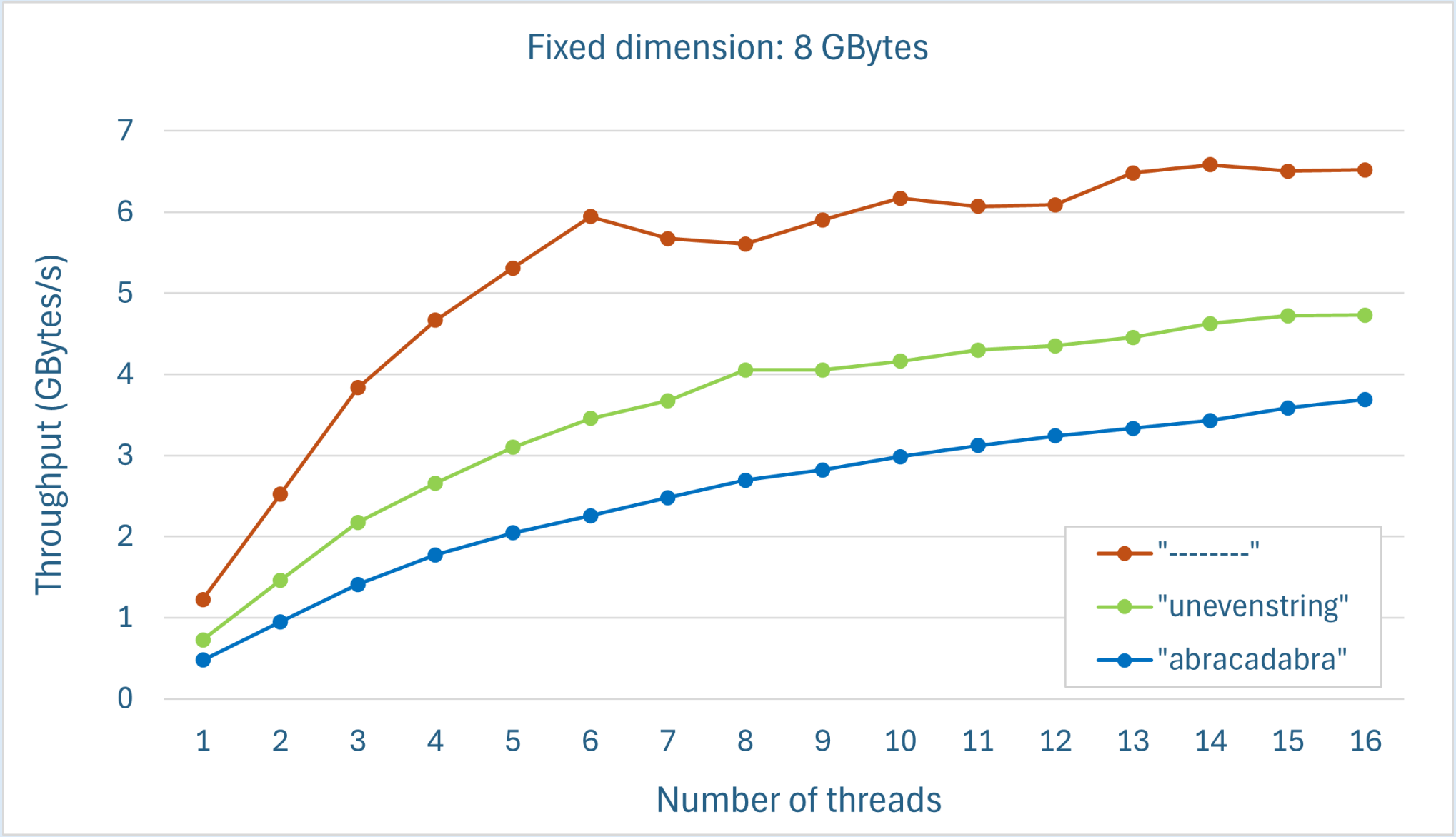
*i = 18*

| B | A | B | A | B | A | B | A | B | C | A | B | A | B | C | A | B | A | B |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

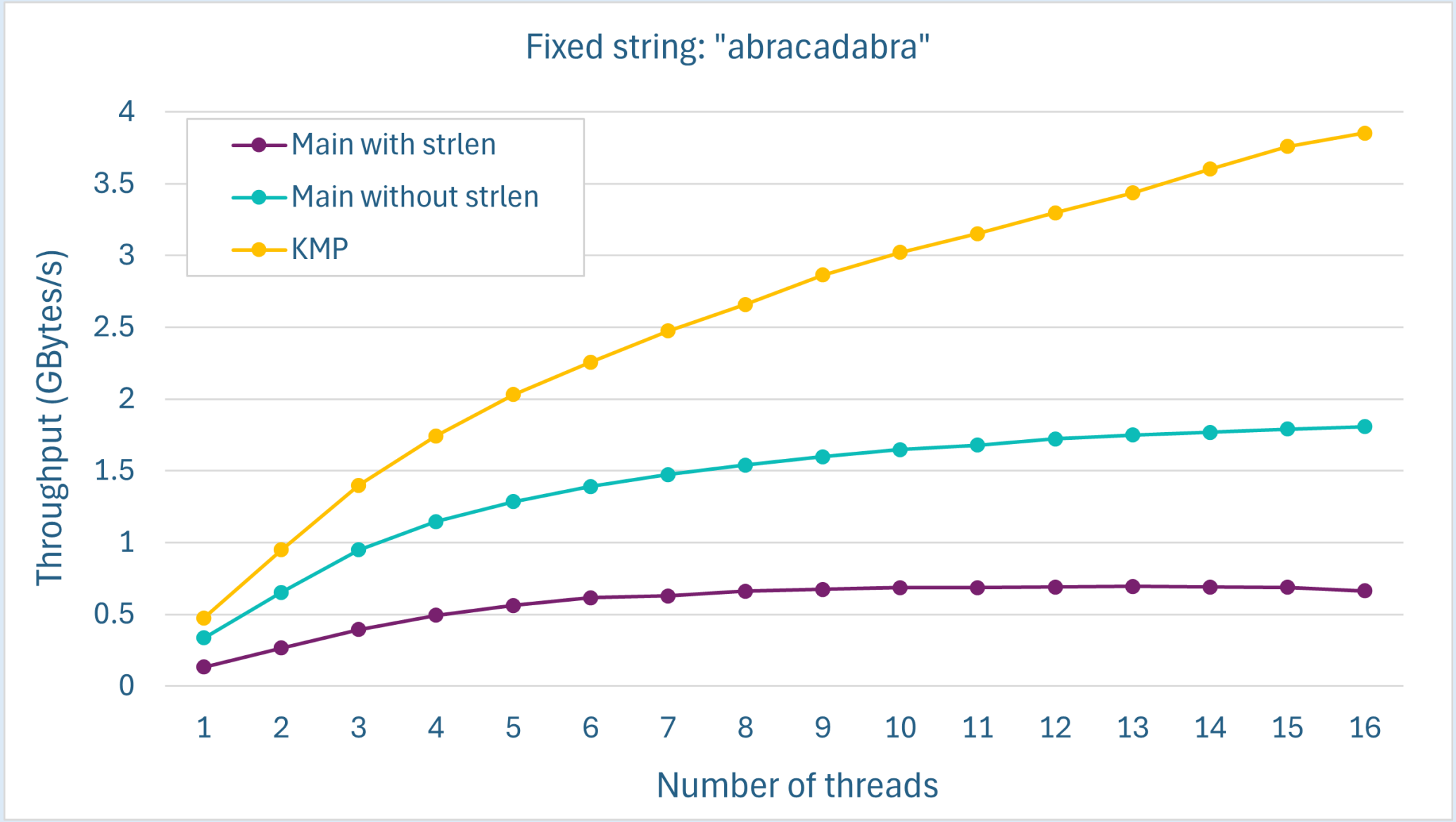
# Throughput analysis: fixed string



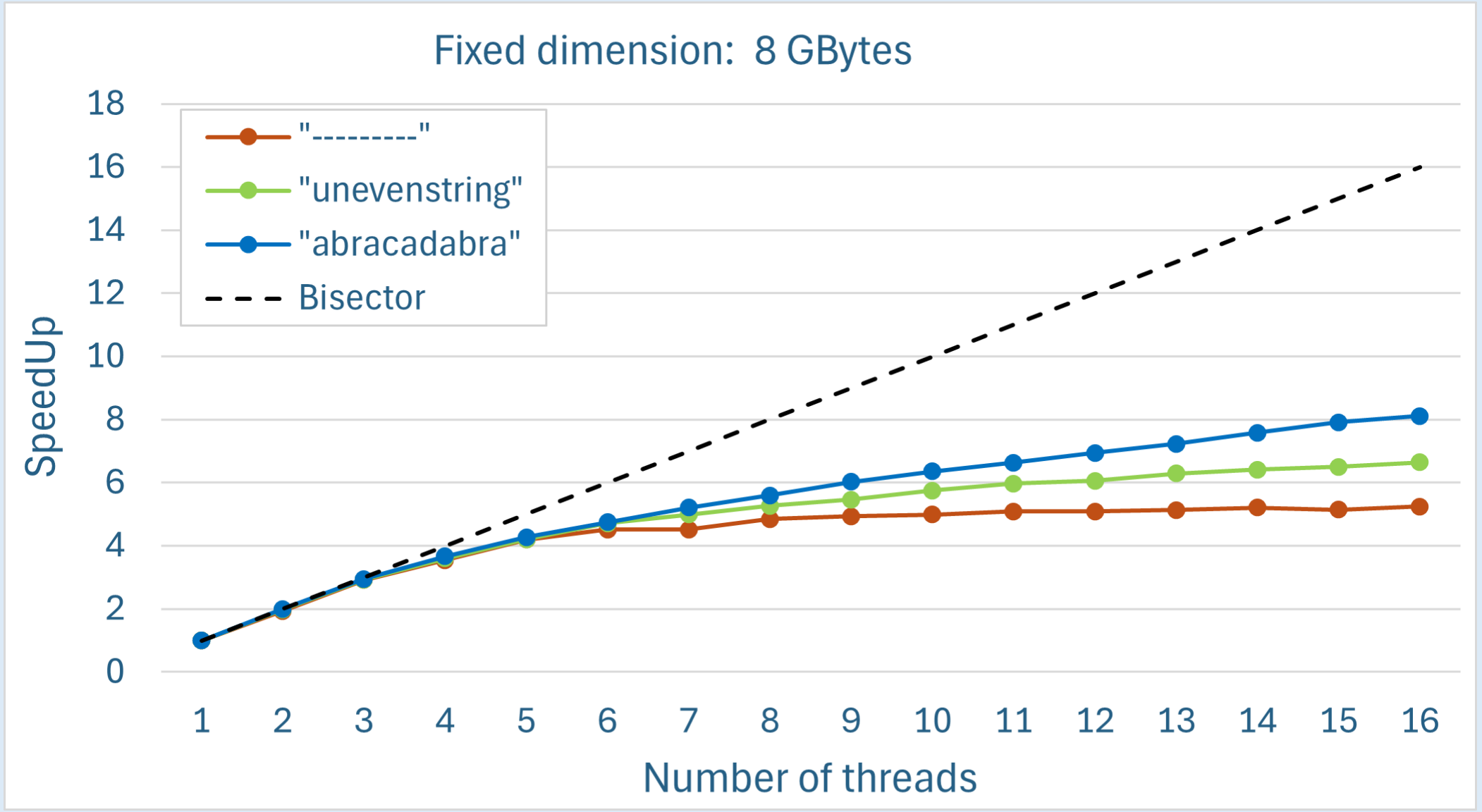
# Throughput analysis: fixed dim



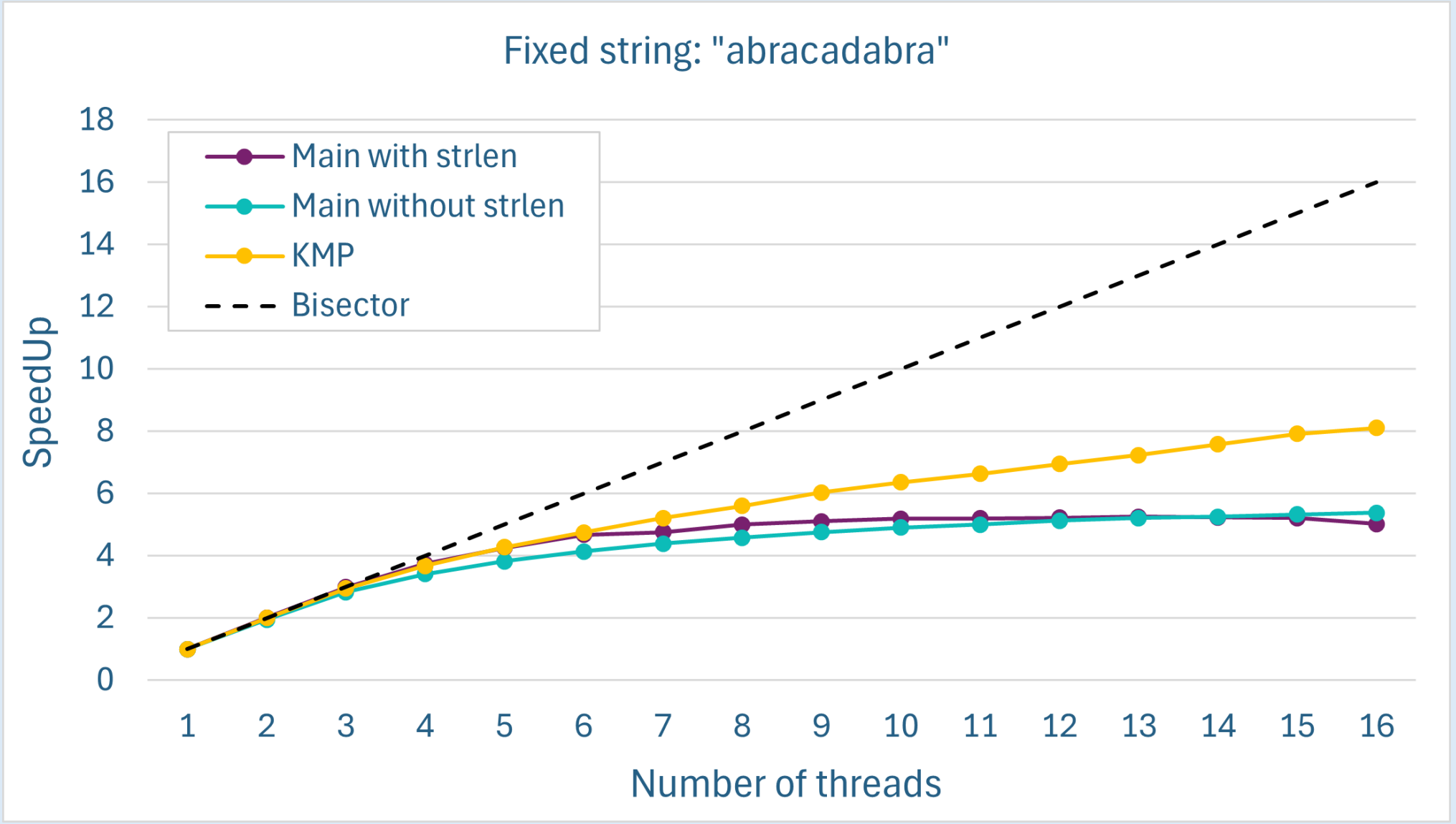
# Throughput analysis: comparison



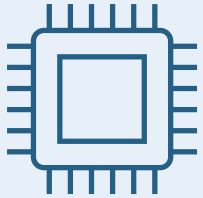
# SpeedUp analysis: fixed dim



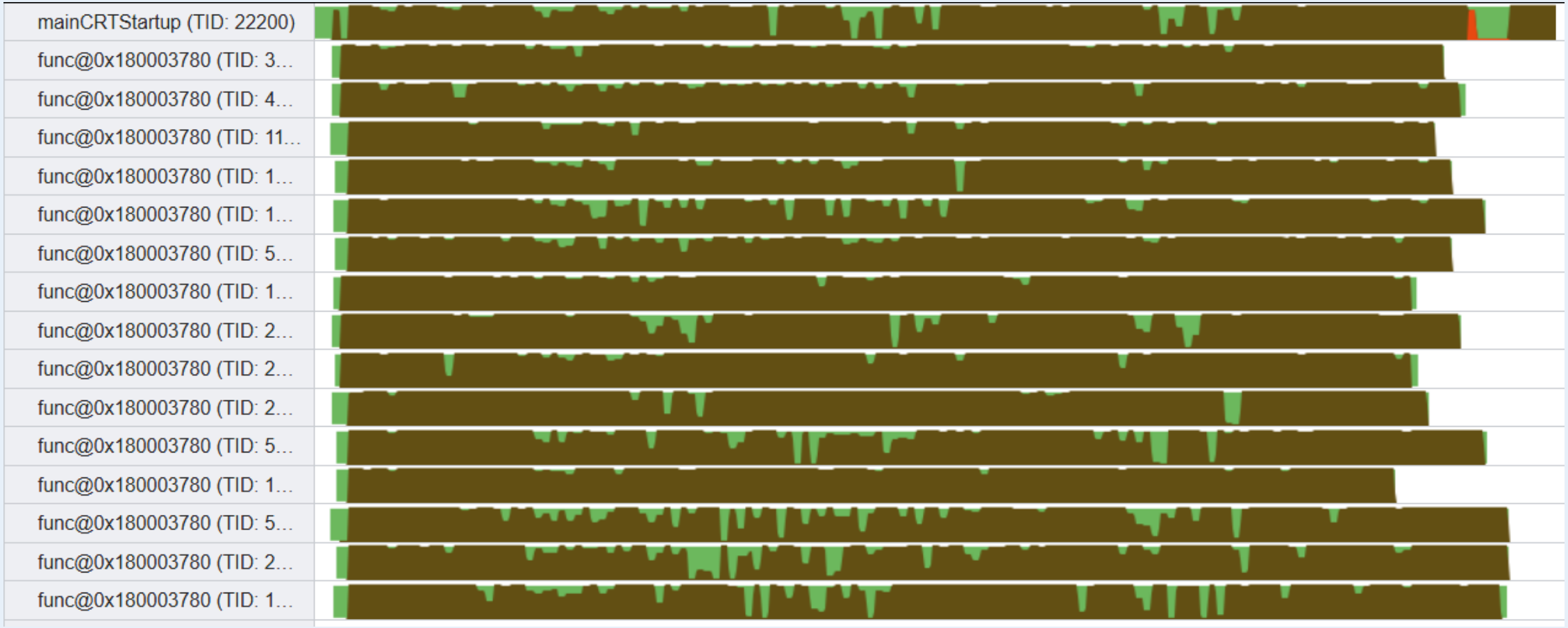
# SpeedUp analysis: comparison



# Is there a way To do even Better?



# KMP with load balancing



# KMP with load balancing

*Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore.*

*First thread*

*Second thread*

*Third thread*

In the previous version of the algorithm, the file was equally distributed among the threads using static partitioning

# KMP with load balancing

*Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore.*

*First thread*

*Second thread*

*Third thread*

Now, the file is divided into fixed-size chunks (5 MB in our case). Threads are dynamically assigned a new chunk as soon as they finish their current one, until the entire file is processed.

# KMP with load balancing

*Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore.*

*Second thread*

*Third thread*

*First thread*

Now, the file is divided into fixed-size chunks (5 MB in our case). Threads are dynamically assigned a new chunk as soon as they finish their current one, until the entire file is processed.

# KMP with load balancing

*Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore.*

*Third thread*

*First thread*

*Second thread*

Now, the file is divided into fixed-size chunks (5 MB in our case). Threads are dynamically assigned a new chunk as soon as they finish their current one, until the entire file is processed.

# KMP with load balancing

*Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore.*

*First thread*

*Second thread*

*Third thread*

Now, the file is divided into fixed-size chunks (5 MB in our case). Threads are dynamically assigned a new chunk as soon as they finish their current one, until the entire file is processed.

# KMP with load balancing

*Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore.*

*Second thread*

*Third thread*

*First thread*

Now, the file is divided into fixed-size chunks (5 MB in our case). Threads are dynamically assigned a new chunk as soon as they finish their current one, until the entire file is processed.

# KMP with load balancing

*Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore.*

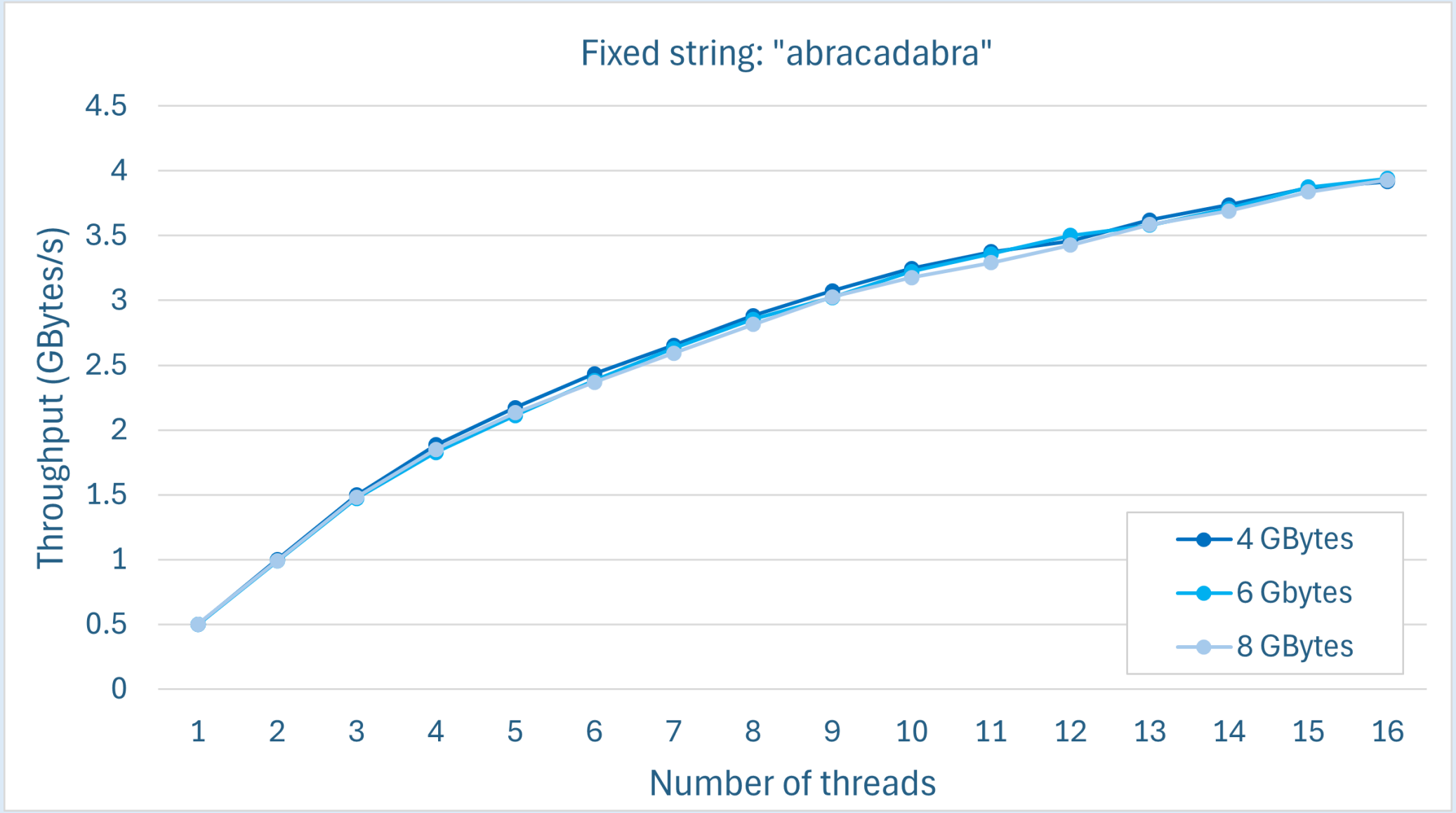
*Third thread*

*First thread*

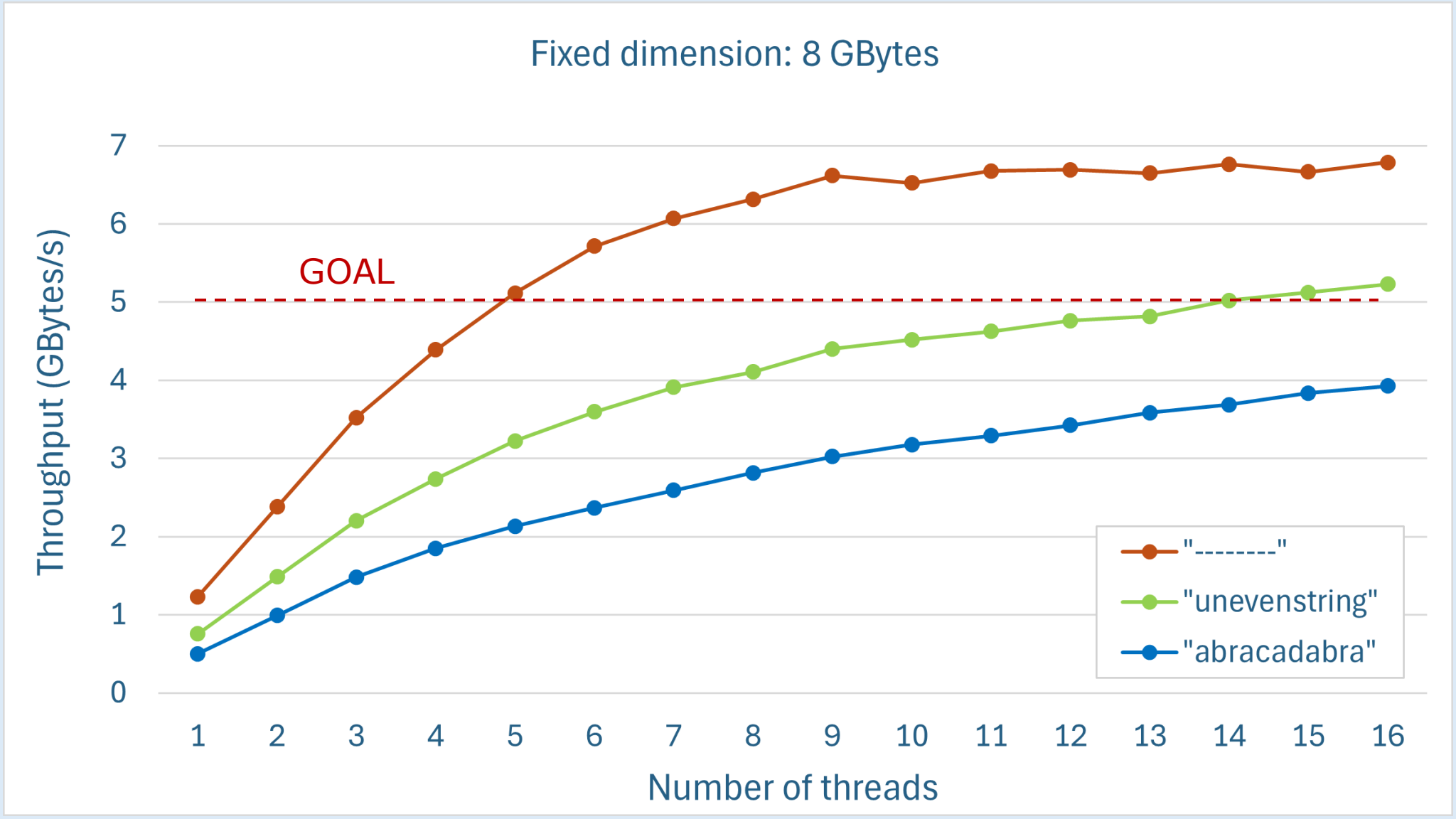
*Second thread*

Now, the file is divided into fixed-size chunks (5 MB in our case). Threads are dynamically assigned a new chunk as soon as they finish their current one, until the entire file is processed.

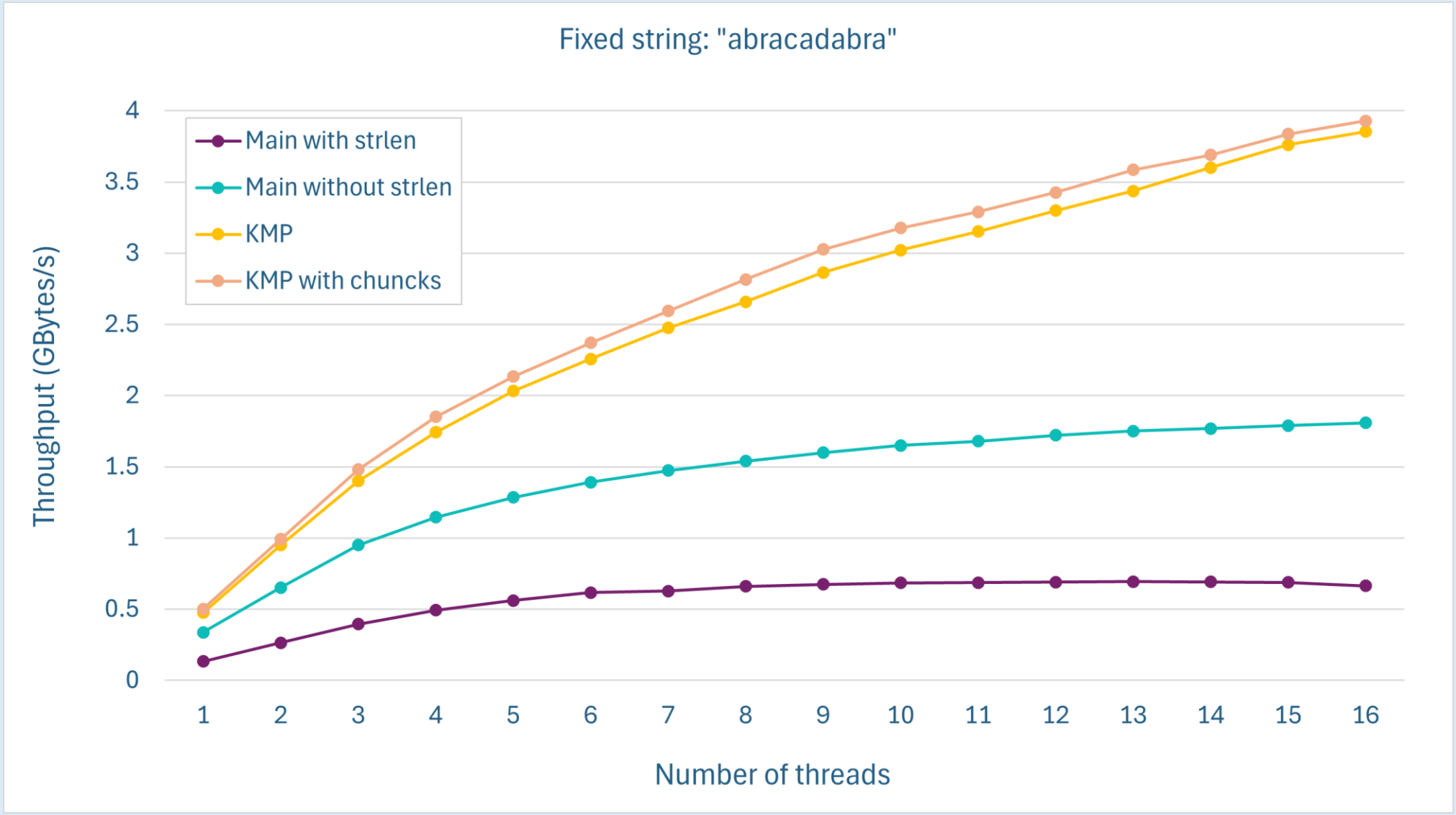
# Throughput analysis: fixed string



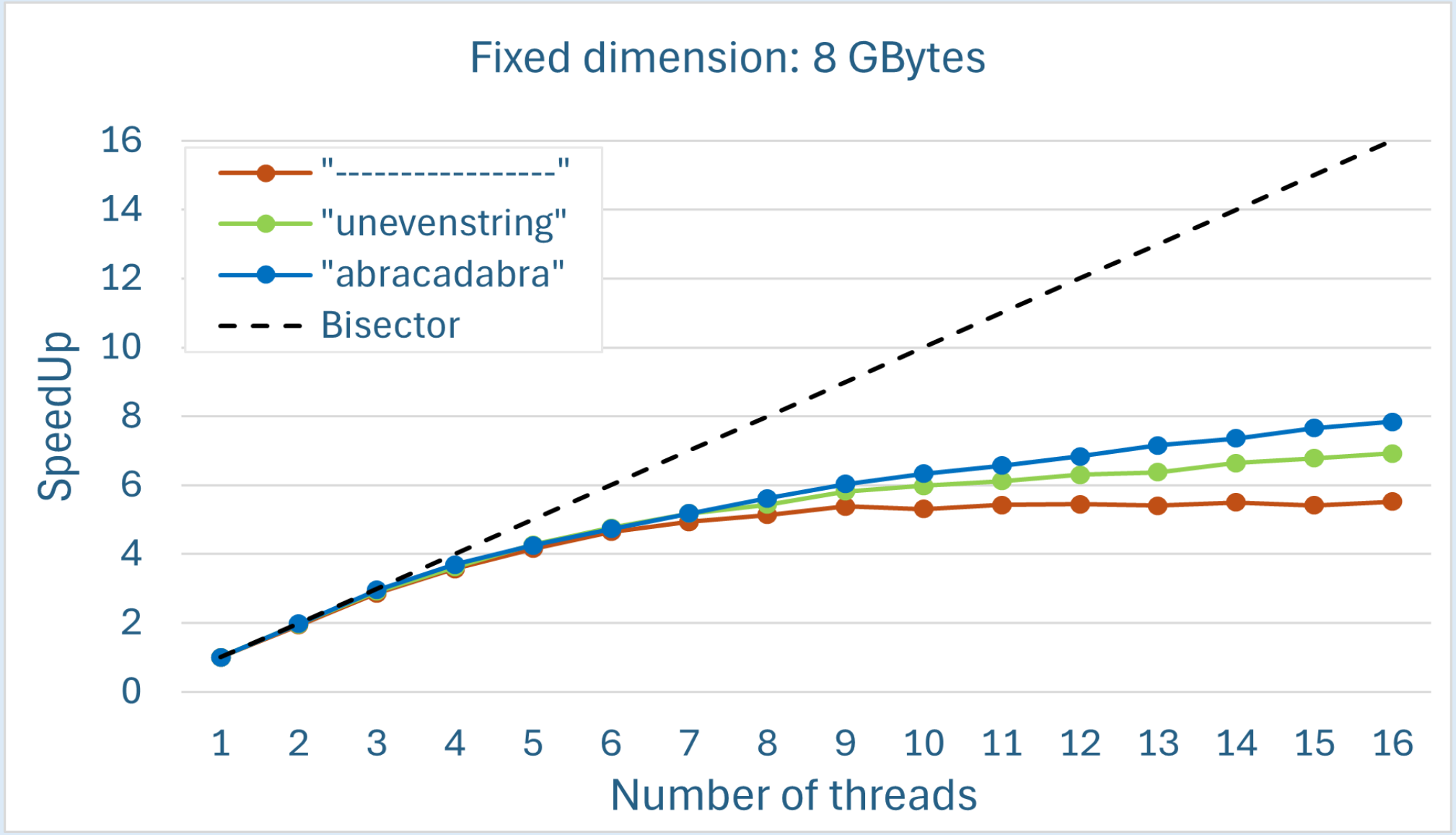
# Throughput analysis: fixed dim



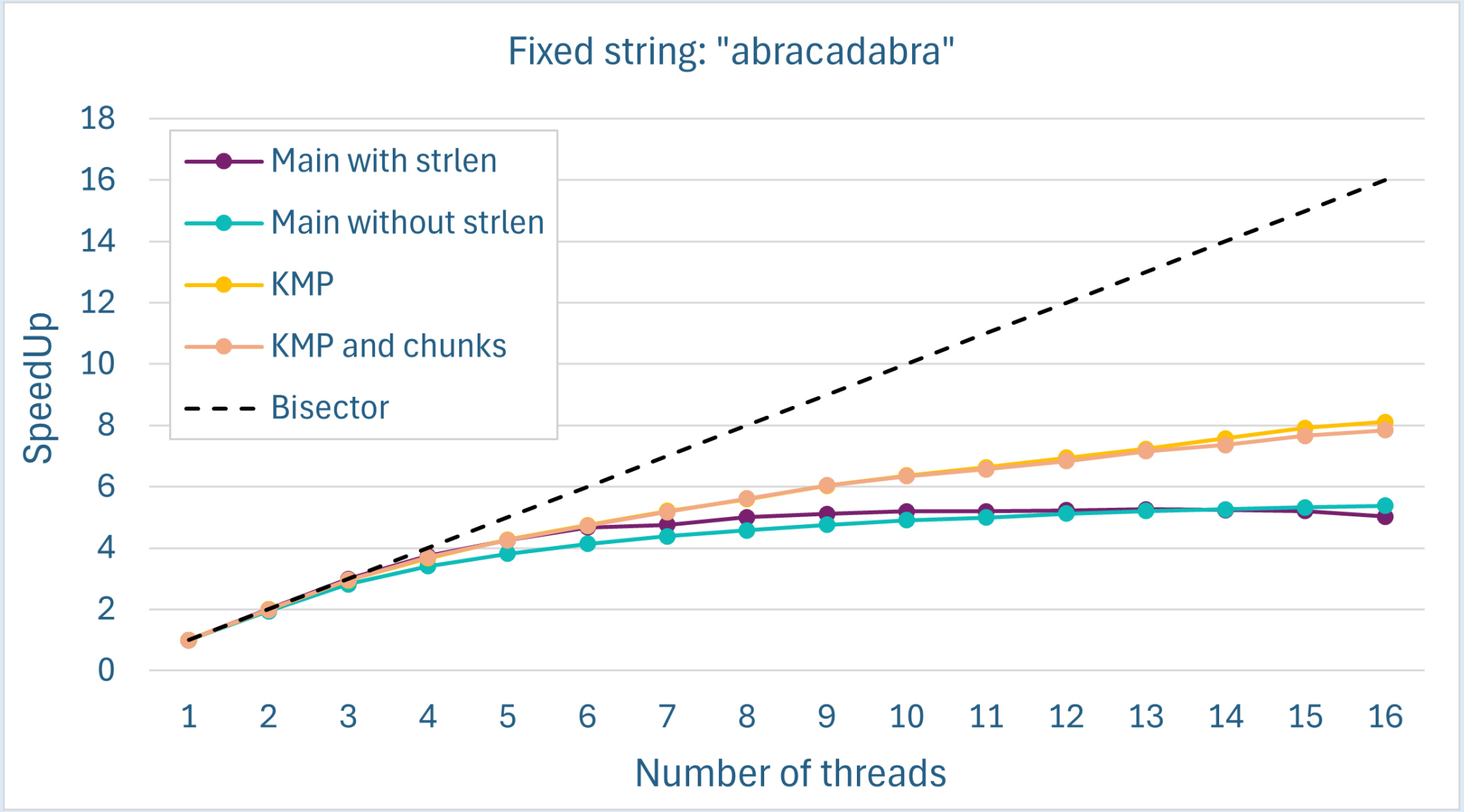
# Throughput analysis: comparison



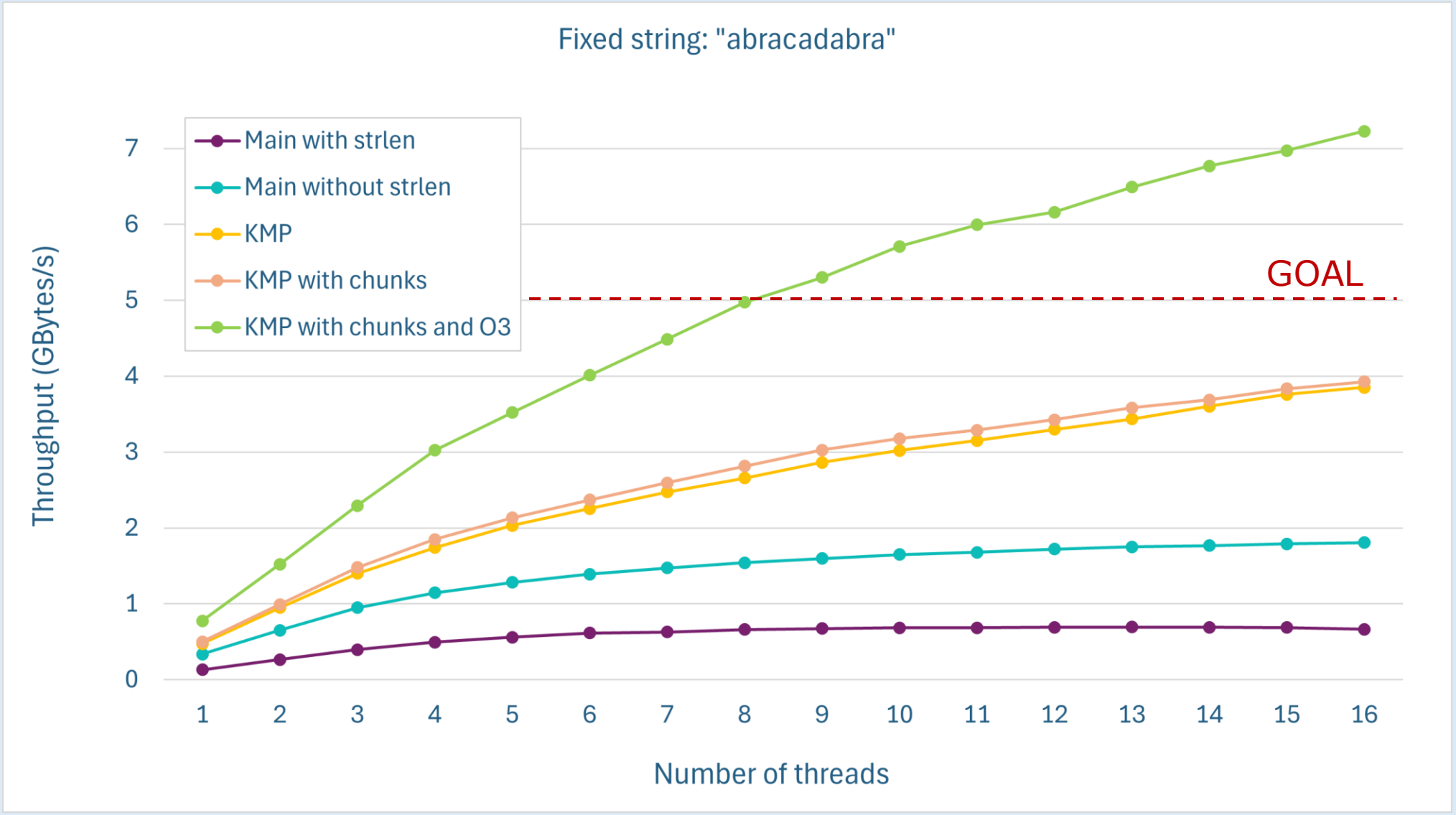
# SpeedUp analysis: fixed dimension



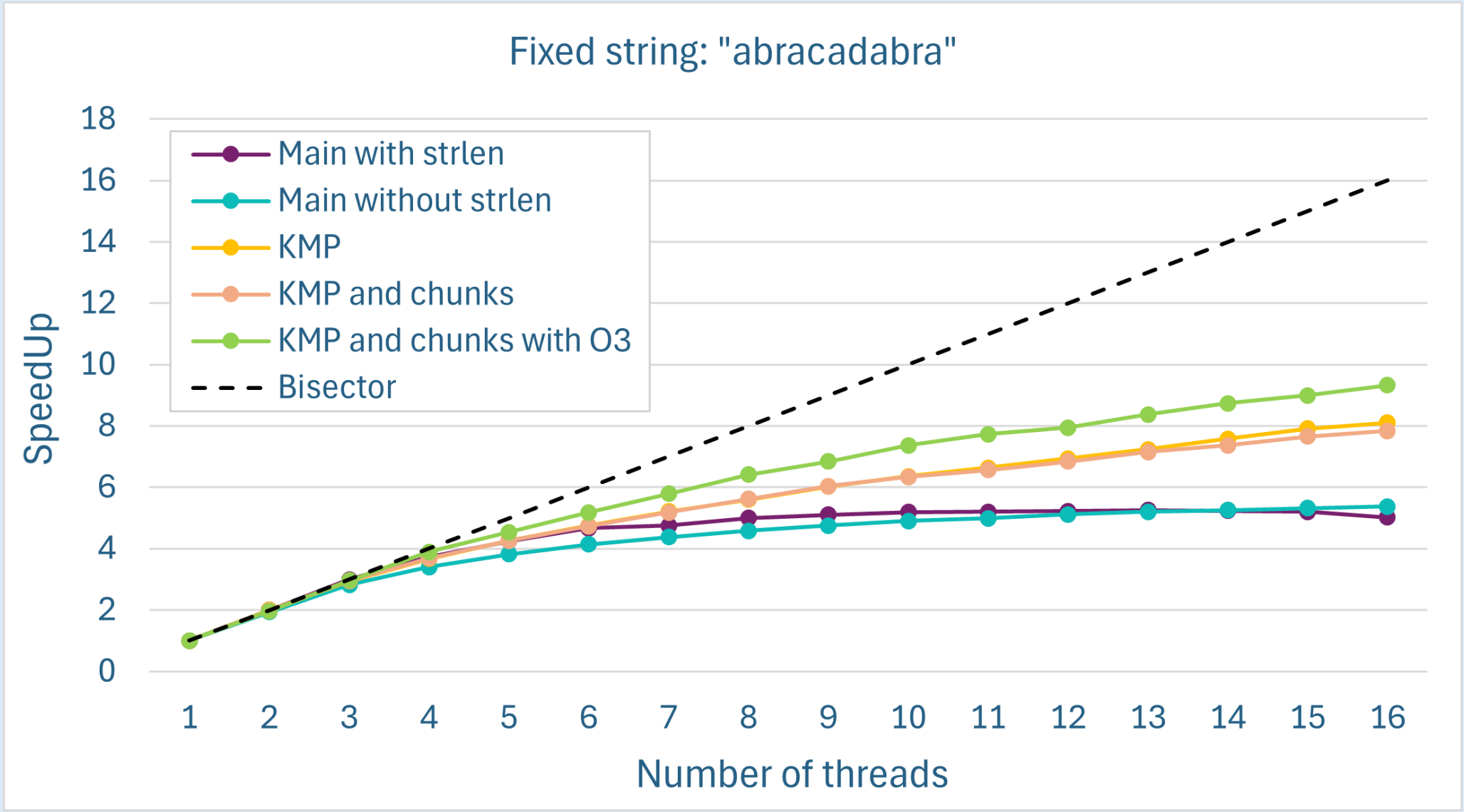
# SpeedUp analysis: comparison



# Throughput analysis: O3 version



# SpeedUp analysis: O3 version




ALGORITHM

GOALS


CPU

GPU

- **To be continued**

Master Degree Computer Engineering
Computer Architecture

# Thanks for the attention

Megan Maremmani
Eleonora Sgorbini
Sebastiano Pala